Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

İteratörler ve STL Algoritmaları

Bölüm 2 – Hafta 2

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 2. Hafta

Bu Haftanın Planı

1. Ünite 1: İteratör Kavramı
2. Ünite 2: Temel STL Algoritmaları
3. Ünite 3: İleri Algoritmalar
4. Ünite 4: Lambda İfadeleri
5. Özet

Ünite 1: İteratör Kavramı

İteratör Nedir?

Tanım

İteratör, kapsayıcıdaki elemanlara sıralı erişim sağlayan ve **akıllı bir işaretçi** gibi davranan nesnedir.

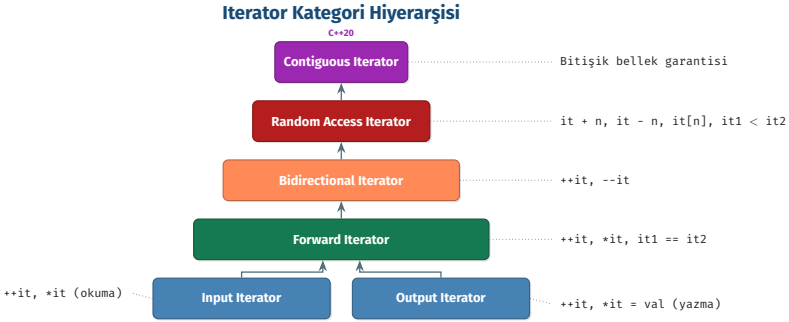
Köprü Metaforu:

- Kapsayıcılar veriyi saklar
- Algoritmalar veri üzerinde işlem yapar
- İteratörler bu ikisini **birbirine bağlar**

Neden Önemli?

5 kapsayıcı + 10 algoritma = **15 bileşen** (iteratör sayesinde)
İteratör olmasa: $5 \times 10 = 50$ özel fonksiyon gerekir!

İteratör Kategorileri



Kategori	Yön	İşlemler	Örnek Kapsayıcı
Input	İleri (tek)	`*it`, `++it`	`istream_iterator`
Output	İleri (tek)	`*it=v`, `++it`	`ostream_iterator`
Forward	İleri	Input + çoklu geçiş	`forward_list`
Bidirectional	İleri/Geri	Forward + `--it`	`list`, `map`, `set`
Random Access	Rastgele	Bidirectional + `it+n`, `it[n]`	`vector`, `deque`
Contiguous	**Rastgele**	**RA + ardışık bellek**	`vector`, `array`, `string`

İteratör Adaptörleri

Ters İteratör:

```
std::vector<int> v = {1,2,3,4,5};
// Sondan basa
for (auto it = v.rbegin();
     it != v.rend(); ++it) {
    std::cout << *it << " ";
}
// Cikti: 5 4 3 2 1
```

Ekleme İteratörü:

```
std::vector<int> hedef;
// Sona ekleyerek kopyala
std::copy(v.begin(), v.end(),
    std::back_inserter(hedef));

// Basa ekle
std::copy(v.begin(), v.end(),
    std::front_inserter(d));
```

Akış İteratörleri

`istream_iterator<T>`: Girişten okuma `ostream_iterator<T>`: Çıkışa yazma

Akış İteratörleri – Örnek

ostream_iterator:

```
#include <iterator>
#include <algorithm>
std::vector<int> v = {1, 2, 3, 4, 5};

// Ekrana yazdir (", " ayirici)
std::copy(v.begin(), v.end(),
    std::ostream_iterator<int>(
        std::cout, ", "));
// Cikti: 1, 2, 3, 4, 5,
```

istream_iterator:

```
// Dosyadan sayilari oku
std::ifstream dosya("notlar.txt");
std::vector<double> notlar(
    std::istream_iterator<double>(dosya),
    std::istream_iterator<double>());

// Ortalama hesapla
double ort = std::accumulate(
    notlar.begin(), notlar.end(), 0.0)
    / notlar.size();
```

Dikkat

Akış iteratörleri **tek geçişlidir** (single-pass). Bir kez okunan/yazılan elemana geri dönülemez.

İteratör Yardımcı Fonksiyonları

```
#include <iterator>
std::vector<int> v = {10, 20, 30, 40, 50};
// distance: iki iterator arasindaki mesafe
auto uzaklik = std::distance(v.begin(), v.end()); // 5
// advance: iteratoru n adim ilerlet (yerinde degistirir)
auto it = v.begin();
std::advance(it, 3);  // it artik 40'i gosterir
// next / prev: yeni iterator dondurur (orijinali degismez)
auto it3 = std::next(v.begin(), 2);  // 30'u gosterir
auto it4 = std::prev(v.end());       // 50'yi gosterir
```

Neden Önemli?

advance ve distance, list gibi rastgele erişim sunmayan kapsayıcılarda da çalışır.
next/prev orijinal iteratörü bozmaz → daha güvenli.

begin/end Ailesi

Fonksiyon	Açıklama	Dönen Tür
`begin()/end()`	Normal iteratör	`iterator`
`cbegin()/cend()`	Sabit (const) iteratör	`const_iterator`
`rbegin()/rend()`	Ters iteratör	`reverse_iterator`
`crbegin()/crend()`	Sabit ters iteratör	`const_reverse_iterator`

```
std::vector<int> v = {10, 20, 30};
auto it = v.begin();    // *it = 10
auto cit = v.cbegin(); // *cit = 10, ama *cit = 5 HATA!

// C++20: std::ssize() - isaretli boyut
for (auto i = 0; i < std::ssize(v); ++i) { ... }
```

Kural

Değiştirmeyeceksen `cbegin()/cend()` tercih et → güvenlik!

Ünite 2: Temel STL Algoritmaları

sort Ailesi

```
#include <algorithm>
std::vector<int> notlar = {85, 92, 78, 95, 60, 88};

// Temel siralama (introsort, O(n log n))
std::sort(notlar.begin(), notlar.end());

// Ozel karsilastirici ile (buyukten kucuge)
std::sort(notlar.begin(), notlar.end(),
          std::greater<int>());

// Kararli siralama (esit elemanlarin sirasi korunur)
std::stable_sort(notlar.begin(), notlar.end());

// Kismi siralama (sadece ilk 3'u sirala)
std::partial_sort(notlar.begin(), notlar.begin() + 3,
                  notlar.end());
```

Performans

sort: $O(n \log n)$ partial_sort: $O(n \log k)$ stable_sort: $O(n \log^2 n)$

find Ailesi

```
std::vector<int> notlar = {60, 78, 85, 88, 92, 95};

// Deger ile arama (O(n))
auto it = std::find(notlar.begin(), notlar.end(), 88);
if (it != notlar.end())
    std::cout << "Bulundu: " << *it << '\n';

// Kosula gore arama
auto it2 = std::find_if(notlar.begin(), notlar.end(),
    [](int n) { return n >= 90; });
// *it2 = 92

// Sirali veri icin ikili arama (O(log n))
bool var = std::binary_search(
    notlar.begin(), notlar.end(), 85); // true
```

Dikkat

`binary_search` sadece **sıralı** veride çalışır! Önce `sort` çağır veya sıralı kapsayıcı kullan.

count, for_each, fill ve generate

```
std::vector<int> notlar = {85, 42, 95, 60, 78, 92};

// count_if: kosulu saglayan eleman sayisi
int basarili = std::count_if(notlar.begin(), notlar.end(),
    [](int n) { return n >= 60; });  // 5

// for_each: her elemana islem uygula (yerinde)
std::for_each(notlar.begin(), notlar.end(),
    [](int& n) { n += 5; });  // Herkese 5 puan eklendi

// fill: tum elemanlari ayni degerle doldur
std::fill(notlar.begin(), notlar.end(), 0);

// generate: her eleman icin fonksiyon cagir
int sayac = 2000;
std::generate(notlar.begin(), notlar.end(),
    [&sayac]() { return ++sayac; });
// notlar: {2001, 2002, 2003, 2004, 2005, 2006}
```

transform ve copy

```
std::vector<int> notlar = {85, 92, 78, 95, 60};
std::vector<double> yuzdeler(notlar.size());

// transform: her elemani donustur
std::transform(notlar.begin(), notlar.end(),
    yuzdeler.begin(),
    [](int n) { return n / 100.0; });
// yuzdeler: {0.85, 0.92, 0.78, 0.95, 0.60}

// copy_if: kosulu saglayanlari kopyala
std::vector<int> basarili;
std::copy_if(notlar.begin(), notlar.end(),
    std::back_inserter(basarili),
    [](int n) { return n >= 80; });
// basarili: {85, 92, 95}
```

transform = map() Dengi

Fonksiyonel programlamadaki `map()` ile aynı mantık!

Remove–Erase Deyimi

Orijinal: 1 | ~~2~~ | 3 | ~~2~~ | 4 | ~~2~~ | 5

size = 7

Hedef: tüm 2'leri sil

`std::remove(v.begin(), v.end(), 2)`

remove: 1 | 3 | 4 | 5 | ? | ? | ?

new_end

belirsiz

size = 7

Boyut değişmedi!

Sadece elemanlar kaydırıldı

`v.erase(new_end, v.end())`

erase: 1 | 3 | 4 | 5

size = 4

Temiz dizi, boyut = 4

C++20: `std::erase(v, 2)` veya `std::erase_if(v, pred)`

Tek satırda aynı işi yapar!

C++20: std::erase ve std::erase_if

```
std::vector<int> notlar = {85, 92, 78, 95, 60, 88};

// C++17 oncesi: remove-erase idiomu
notlar.erase(
    std::remove(notlar.begin(), notlar.end(), 78),
    notlar.end());

// C++20: Tek satirda!
std::erase(notlar, 78);

// C++20: Kosullu silme
std::erase_if(notlar,
    [](int n) { return n < 80; });
// notlar: {85, 92, 95, 88}
```

C++20 Avantajı

Daha temiz, daha okunabilir, hata riski düşük!

Ünite 3: İleri Algoritmalar

accumulate ve reduce

```
#include <numeric>
std::vector<double> notlar = {3.5, 2.8, 3.9, 3.2};

// accumulate: soldan saga birikimli islem
double toplam = std::accumulate(
    notlar.begin(), notlar.end(), 0.0);
double ort = toplam / notlar.size(); // GPA hesaplama!

// Ozel islem ile carpim
double carpim = std::accumulate(notlar.begin(),
    notlar.end(), 1.0, std::multiplies<double>());

// C++17 reduce: paralel calisabilir
double top2 = std::reduce(
    notlar.begin(), notlar.end(), 0.0);
```

Dikkat: Başlangıç Değeri

`accumulate(..., 0)` ile `accumulate(..., 0.0)` farklı! 0 → `int` sonuç, 0.0 → `double` sonuç.

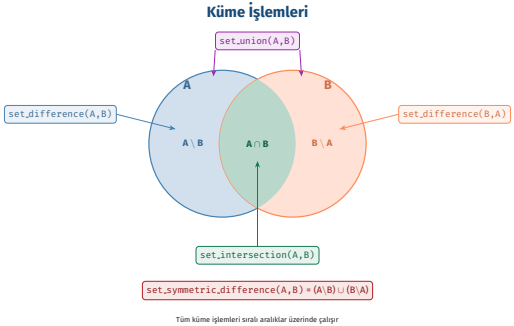
inner_product, partial_sum ve transform_reduce

```
#include <numeric>
// OBYS: Agirlikli GNO hesaplama
std::vector<int> krediler = {3, 4, 3, 2};
std::vector<double> notlar = {3.5, 2.8, 3.9, 4.0};

// inner_product: kredi * not toplamini hesapla
double agirlikli = std::inner_product(
    krediler.begin(), krediler.end(), notlar.begin(), 0.0);
double gno = agirlikli / std::accumulate(
    krediler.begin(), krediler.end(), 0);

// partial_sum: kumulatif kredi toplami
std::vector<int> kumulatif(krediler.size());
std::partial_sum(krediler.begin(), krediler.end(),
    kumulatif.begin());  // {3, 7, 10, 12}
```

C++17: transform_reduce

`transform_reduce`: Dönüştür + biriktir tek geçişte. `reduce`'un jenerik hali, paralel çalışabilir.

Küme İşlemleri




```
// OBYS: Ortak ders bulma
std::set<std::string> ali =
    {"Mat","Fiz","C++"};
std::set<std::string> ayse =
    {"Mat","Kim","C++"};

std::vector<std::string> ortak;
std::set_intersection(
    ali.begin(), ali.end(),
    ayse.begin(), ayse.end(),
    std::back_inserter(ortak));
// ortak: {"C++", "Mat"}
```

Diğer Küme İşlemleri

`set_union` (birleşim) `set_difference` (fark) `set_symmetric_difference` (simetrik fark)

Not: Girdi aralıkları **sıralı** olmalıdır!

partition ve nth_element

```
std::vector<int> notlar = {85, 42, 95, 60, 78, 92};

// partition: kosulu saglayanlari one tasir
auto sinir = std::partition(
    notlar.begin(), notlar.end(),
    [](int n) { return n >= 70; });
// [85, 92, 95, 78 | 60, 42]
//                 ^ sinir

// stable_partition: goreli sirayi korur
std::stable_partition(notlar.begin(), notlar.end(),
    [](int n) { return n >= 70; });

// nth_element: n. eleman dogru konuma gelir
std::vector<int> v = {5, 1, 9, 3, 7, 2, 8};
std::nth_element(v.begin(), v.begin() + 3, v.end());
// v[3] = 5 (medyan degeri dogru konumda)
```

Performans

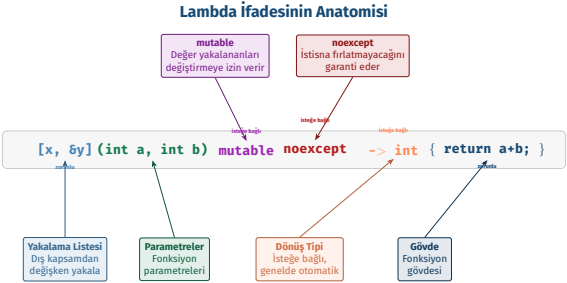
partition: $O(n)$ nth_element: $O(n)$ ortalama

OBYS İstatistik Örneği

```
struct Ogrenci { std::string ad; double gpa; };
std::vector<Ogrenci> liste = {
    {"Ali", 3.5}, {"Ayse", 2.8}, {"Mehmet", 3.9},
    {"Zeynep", 1.9}, {"Can", 3.2}, {"Elif", 2.5}
};

// 1. GPA'ya gore sirala
std::sort(liste.begin(), liste.end(),
    [](const auto& a, const auto& b) {
        return a.gpa > b.gpa; });

// 2. Ortalama GPA
double ort = std::accumulate(liste.begin(), liste.end(),
    0.0, [](double t, const auto& o) {
        return t + o.gpa; }) / liste.size();

// 3. Basarili/basarisiz ayir
auto sinir = std::partition(liste.begin(), liste.end(),
    [](const auto& o) { return o.gpa >= 2.0; });
```

Ünite 4: Lambda İfadeleri

Lambda Anatomisi

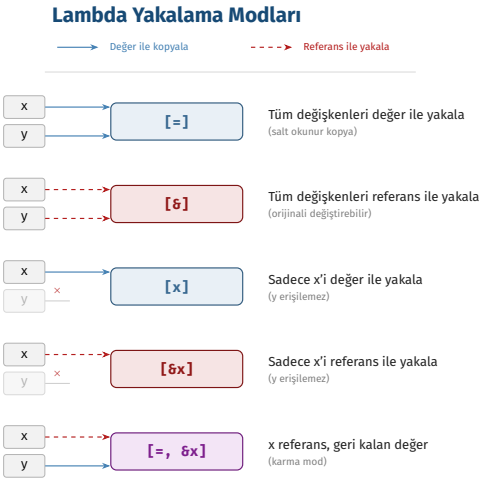


Söz Dizimi

```
[yakalama](parametreler) -> dönüş_türü { gövde }
```

- **Yakalama**: Dış kapsamdaki değişkenlere erişim
- **Parametreler**: Normal fonksiyon parametreleri
- **Dönüş türü**: Genellikle otomatik çıkarılır
- **Gövde**: Çalışacak kod bloğu

Yakalama Modları



Söz Dizimi	Açıklama
`[]`	Hiçbir şey yakalama
`[x]`	`x`'i değerle yakala
`[&x]`	`x`'i referansla yakala
`[=]`	Hepsini değerle yakala
`[&]`	Hepsini referansla yakala
`[=, &x]`	Hepsi değer, `x` referans
`[&, x]`	Hepsi referans, `x` değer
`[this]`	`this` işaretçisini yakala
`[*this]`	Nesne kopyasını yakala (C++17)

mutable, constexpr ve IIFE

mutable Lambda:

```
int sayac = 0;
auto artir = [sayac]() mutable {
    return ++sayac;
    // Kopya uzerinde degisiklik
};
artir(); // 1
artir(); // 2
// sayac hala 0!
```

IIFE Deseni:

```
// Karmasik const baslangic
const auto tablo = []() {
    std::map<std::string, double> m;
    m["AA"] = 4.0;
    m["BA"] = 3.5;
    m["BB"] = 3.0;
    m["CC"] = 2.0;
    m["FF"] = 0.0;
    return m;
}(); // () hemen cagiriyor!
```

IIFE: Immediately Invoked Function Expression

Lambda tanımlayıp **hemen çağırmak** → karmaşık `const` başlatmalarında çok kullanışlı.
Sondaki () parantezine dikkat!

Init Capture ve Taşıma (C++14)

```
// Init capture: lambda icinde yeni degisken olustur
auto sayac = [n = 0]() mutable { return ++n; };
sayac(); // 1
sayac(); // 2
// Baslangic ifadesi ile hesaplama
auto filtre = [esik = ortalama * 1.2](double not_) {
    return not_ >= esik;
};
// unique_ptr'yi lambda'ya TASIMA
auto ogr = std::make_unique<Ogrenci>("Ali", 1001);
// [ogr] HATA! unique_ptr kopyalanamaz
auto lambda = [ogr = std::move(ogr)]() {
    std::cout << ogr->isim << '\n';
};
```

Neden Önemli?

Init capture, `std::move` ile taşınabilir türleri (unique_ptr, thread) lambda'ya aktarmanın **tek yoludur**.

[this] vs [*this] ve Sarkan Referans

[this] – İşaretçi yakalar:

```
class Rapor {
    std::string baslik;
public:
    auto al_lambda() {
        return [this]() {
            // this silindiyse -> COKME!
            return baslik;
        };
    }
};

auto f = rapor.al_lambda();
// rapor yok edilirse f tehlikeli!
```

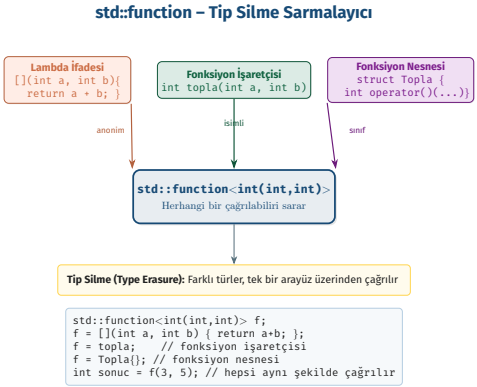
[*this] – Kopya yakalar (C++17):

```
class Rapor {
    std::string baslik;
public:
    auto al_lambda() {
        return [*this]() {
            // Nesne kopyasi guvenli
            return baslik;
        };
    }
};

auto f = rapor.al_lambda();
// rapor yok edilse bile f guvenli!
```

Sarkan Referans Tehlikesi

`[&]` ile yakalanan yerel değişken kapsam dışına çıkarsa **tanımsız davranış**! Asenkron işlemlerde ve Qt slot'larında `[*this]` veya değer yakalama tercih edin.

std::function




```cpp
#include <functional>
// Her turlu cagrilabiliri sarar
std::function<bool(int)> filtre;
// Lambda atama
filtre = [](int n) {
    return n >= 80;
};
// Normal fonksiyon atama
bool kontrol(int n) {
    return n >= 80;
}
filtre = kontrol;
```

Performans Notu

`std::function` tür silme (type erasure) kullanır → küçük bir çalışma zamanı maliyeti var. Mümkünse `auto` tercih edin.

Jenerik Lambda ve Template Lambda

C++14: Jenerik Lambda

```
// auto parametre = template
auto yazdir = [](const auto& x) {
    std::cout << x << '\n';
};

yazdir(42);         // int
yazdir(3.14);       // double
yazdir("Merhaba"); // const char*
```

C++20: Template Lambda

```
// Acik template parametresi
auto topla = []<typename T>
    (const std::vector<T>& v) {
    return std::accumulate(
        v.begin(), v.end(), T{});
};

std::vector<int> vi = {1,2,3};
std::vector<double> vd = {1.1,2.2};
topla(vi); // 6
topla(vd); // 3.3
```

Evrim

C++11: Lambda → C++14: Jenerik lambda → C++17: constexpr → C++20: Template lambda

Birleştirilebilir Yüklemler (Composable Predicates)

```
// Temel filtreler (lambda fabrikasi)
auto aktif = [](const Ogrenci& o) { return o.aktif; };
auto yuksek_not = [](const Ogrenci& o) { return o.gpa >= 3.5; };

// NOT: yuklemi tersine cevir
auto degil = [](auto y) {
    return [y](const auto& x) { return !y(x); }; };

// AND: iki yuklemi birlestir
auto ve = [](auto y1, auto y2) {
    return [y1,y2](const auto& x) { return y1(x) && y2(x); }; };

// OR: iki yuklemden birini karsilayanlar
auto veya = [](auto y1, auto y2) {
    return [y1,y2](const auto& x) { return y1(x) || y2(x); }; };

// OBYS: aktif VE (yuksek notlu VEYA bilgisayarci)
auto sorgu = ve(aktif, veya(yuksek_not, bilgisayar));
auto n = std::ranges::count_if(ogrenciler, sorgu);
```

Özet

Bu Hafta Öğrendiklerimiz

İteratörler:

1. 6 kategori (Input → Contiguous)
2. Adaptörler: reverse, insert, stream
3. Yardımcılar: `distance`, `next`, `prev`

Temel Algoritmalar:

4. sort, find, binary_search
5. count_if, for_each, transform
6. Remove-Erase ve C++20 erase_if

İleri Algoritmalar:

7. accumulate, reduce, inner_product
8. partial_sum, transform_reduce
9. Küme işlemleri, partition

Lambda İfadeleri:

10. Söz dizimi ve yakalama modları
11. Init capture, IIFE, [*this]
12. std::function, jenerik lambda
13. Birleştirilebilir yüklemler

OBYS'de İteratör + Algoritma + Lambda

```
#include <algorithm>
#include <numeric>
#include <vector>
#include <string>

struct Ogrenci { std::string ad; double gpa; };

void obysRapor(std::vector<Ogrenci>& liste) {
    // 1. GPA'ya gore sirala (lambda)
    std::sort(liste.begin(), liste.end(),
        [](const auto& a, const auto& b) { return a.gpa > b.gpa; });

    // 2. Ortalama hesapla (accumulate + lambda)
    double ort = std::accumulate(liste.begin(), liste.end(), 0.0,
        [](double t, const auto& o) { return t + o.gpa; }) / liste.size();

    // 3. Basarisizlari kaldir (C++20 erase_if + lambda)
    std::erase_if(liste, [](const auto& o) { return o.gpa < 2.0; });
}
```

Üç Temel Bileşen Bir Arada

`sort` + `accumulate` + `erase_if`: Lambda ile davranış kazandırılan algoritmalar, iteratörler aracılığıyla kapsayıcı üzerinde çalışır.

Gelecek Hafta: Modern C++ ve STL Uygulamaları

- `std::optional`, `std::variant`, `std::any`
- `std::string_view` ve bellek verimliliği
- `std::format` ile modern biçimlendirme
- C++20 Ranges ve Views
- OBYS: Modern C++ dönüşümü – güvenli ve okunabilir kod

Hazırlık

Kitabın Bölüm 3'ü okuyun. Lab02'deki lambda ve algoritma örneklerini OBYS projenize entegre edin.